

# P3450R0

## #embed offset parameter

Published Proposal, 2024-12-13

### Authors:

[JeanHeyd Meneide](#)

[Shepherd \(Shepherd's Oasis LLC\)](#)

### Latest:

[https://thephd.dev/\\_vendor/future\\_cxx/papers/d3540.html](https://thephd.dev/_vendor/future_cxx/papers/d3540.html)

### Paper Source:

[GitHub ThePhD/future\\_cxx](#)

### Implementation:

[GitHub ThePhD/embed](#)

### Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

### Audience:

EWG

---

## Abstract

An additional, user-supported embed parameter implemented in Clang and GCC for providing an offset.

## Table of Contents

- 1 Changelog**
  - 1.1 Revision 0 - December 13<sup>th</sup>, 2024
- 2 Introduction and Motivation**
- 3 Wording**

- 3.1 Intent
- 3.2 Proposed Language Wording
  - 3.2.1 Add to the *control-line* production in §15.1 Preamble [**cpp.pre**] a new grammar production for **offset**
  - 3.2.2 Add a new sub-clause §15.4.2. ✨ under Resource Inclusion for Embed parameters for the new **offset** parameter [**cpp.res.param.offset**]
- 3.3 Add a new example to the **if\_empty** embed parameter [cpp.res.if\_empty] section

## § 1. Changelog

### § 1.1. Revision 0 - December 13<sup>th</sup>, 2024

— Initial release.

## § 2. Introduction and Motivation

The goal is to add the extremely-popular and already-implemented **gnu::offset** and **clang::offset** parameters as standard parameters. That is the only motivation of this proposal; to standardize existing practice.

Originally, users asked to add this parameter, but only after C23 standardized. Given the late stage that users have asked -- waiting until the very end -- it has to be added separately. This proposal aims to standardize what users have asked for, and what Clang and GCC have implemented.

## § 3. Wording

This wording is relative to C++'s latest working draft.

### § 3.1. Intent

The intent of the wording is to provide a preprocessing directive that:

- allows an **offset** parameter skipping over a number of elements before beginning to write out elements to the comma-delimited list.

## § 3.2. Proposed Language Wording

- § 3.2.1. Add to the *control-line* production in §15.1 Preamble [cpp.pre] a new grammar production for **offset**

*embed-standard-parameter:*

**limit** ( *pp-balanced-token-seq* )

**offset** ( *pp-balanced-token-seq* ).

**prefix** ( *pp-balanced-token-seq<sub>opt</sub>* )

**suffix** ( *pp-balanced-token-seq<sub>opt</sub>* )

**if\_empty** ( *pp-balanced-token-seq<sub>opt</sub>* )

§ 3.2.2. Add a new sub-clause §15.4.2. ✨ under Resource Inclusion for Embed parameters for the new **offset** parameter [cpp.res.param.offset]

15.4.2. ✨ **offset parameter**

[cpp.res.param.offset]

- 1 An *embed-parameter* of the form `offset ( pp-balanced-token-seq )` denotes the maximum number of elements that are produced in the comma-delimited list. It shall appear at most once in the *embed-parameter-seq*.
- 2 The *pp-balanced-token-seq* is evaluated as a *constant-expression* using the rules as described in conditional inclusion ([[cpp.cond](#)]), but without being processed as in normal text an additional time.
- 3 The *constant-expression* shall be an integral constant expression whose value is greater than or equal to zero. It shall provide the value for *resource-offset*. The `embed` directive performs *resource-offset* consecutive calls to `std::fgetc` ([[cstdio.syn](#)]) from the resource, as a file. If a call to `std::fgetc` returns EOF, the program is ill-formed. Otherwise, the result of the call is discarded. Each call to `std::fgetc` does not count towards the number of consecutive calls for *resource-count*.

[Example:

```
constexpr const unsigned char sound_signature[] = {
// a hypothetical resource
#embed <sdk/jump.wav> limit(2+2)
};

constexpr const unsigned char truncated_sound_signature[] = {
// a hypothetical resource
#embed <sdk/jump.wav> offset(2) limit(2)
};

// verify PCM WAV resource
static_assert(sizeof(sound_signature) == 4);
static_assert(sound_signature[0] == 'R');
static_assert(sound_signature[1] == 'I');
static_assert(sound_signature[2] == 'F');
static_assert(sound_signature[3] == 'F');
```

```
static_assert(sizeof(truncated_sound_signature) == 2);
static_assert(sound_signature[0] == 'F');
static_assert(sound_signature[1] == 'F');
```

expands to:

```
constexpr const unsigned char sound_signature[] = {
// a hypothetical example
(unsigned char)0x52,
(unsigned char)0x49,
(unsigned char)0x46,
(unsigned char)0x46,
};

constexpr const unsigned char truncated_sound_signature[] = {
(unsigned char)0x46,
(unsigned char)0x46,
};

// verify PCM WAV resource
static_assert(sizeof(sound_signature) == 4);
static_assert(sound_signature[0] == 'R');
static_assert(sound_signature[1] == 'I');
static_assert(sound_signature[2] == 'F');
static_assert(sound_signature[3] == 'F');

static_assert(sizeof(truncated_sound_signature) == 2);
static_assert(sound_signature[0] == 'F');
static_assert(sound_signature[1] == 'F');
```

– end example]

### § 3.3. Add a new example to the `if_empty` embed parameter [cpp.res.if\_empty] section

[*Example:* Given a resource `<single_byte>` that has a *resource-count* of 1, the following directives:

```
#embed <single_byte> offset(1) if_empty(44203)
#embed <single_byte> limit(0) offset(1) if_empty(44203)
```

are replaced with:

```
42203
42203
```

– *end example*]